

WOXSEN UNIVERSITY

School of Technology

DBMS COURSE PROJECT REPORT

CINEMA SCREENING AND TICKET BOOKING SYSTEM

Design and Implementation of a Database Management System (CinemaDB)

Submitted by

Name	Roll Number
T.Harshitha Rani	25WU0102292
B . Manvitha	25WU0102322
V. Dikshit Reddy	25WU0101152
CH . Ameydeep	25WU0101012

Course	Faculty	Academic Year
Database Management Systems (DBMS)	Mrs. Kiran mayee	2026–2027

CERTIFICATE

This is to certify that the project entitled “**CINEMA SCREENING AND TICKET BOOKING SYSTEM**” has been successfully completed and submitted by:

T.Harshitha Rani	25WU0102292
B . Manvitha	25WU0102322
V. Dikshit Reddy	25WU0101152
CH . Ameydeep	25WU0101012

in partial fulfillment of the requirements of the **Database Management Systems (DBMS)** course of the School of Technology, Woxsen University, during the academic year **2026–2027**.

The project demonstrates the application of relational database concepts, SQL commands, database normalization, entity relationship modeling, data manipulation, query processing and user interface design. The work has been carried out under the guidance and supervision of [**Faculty Name**].

Faculty Signature	Department	Date	Place
Mrs. Kiran mayee	School of Technology		Woxsen University

DECLARATION

We hereby declare that the project titled “**Cinema Screening and Ticket Booking System**” is our original work carried out as part of the Database Management Systems course.

The project has been developed to demonstrate our understanding of database design, relational schemas, SQL commands, normalization, entity relationship modeling, database queries and user interface design. The database structure, sample data, SQL commands and queries used in this project have been developed for academic purposes.

We further declare that this project has not been submitted previously for the award of any other academic qualification.

Team Member	Name	Roll No.	Signature
	T.Harshitha Rani	25WU0102292	
	B . Manvitha	25WU0102322	
	V. Dikshit Reddy	25WU0101152	
	CH . Ameydeep	25WU0101012	

Date: _____

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported us during the development of the **Cinema Screening and Ticket Booking System** project.

First, we thank our faculty member [**Faculty Name**] for giving us the opportunity to work on this project and for the valuable guidance provided throughout its development.

We are grateful for the knowledge gained in the Database Management Systems course. The concepts of relational databases, SQL, keys, constraints, normalization, relationships, joins, aggregate functions and database design were essential for completing this project.

We also thank the School of Technology and Woxsen University for providing the academic environment and resources required for the work. We appreciate the contributions of all three team members in designing the database, preparing the SQL commands, developing the queries, designing the user interface, testing the system and preparing the documentation.

Finally, we thank our friends and classmates for their suggestions and feedback during development and testing. This project helped us understand how theoretical DBMS concepts can be applied to a practical, real-world problem such as cinema ticket booking.

ABSTRACT

The **Cinema Screening and Ticket Booking System** is a relational database project developed to manage the operations of a cinema in an organized and efficient manner. In a traditional cinema environment, information about movies, screens, seats, customers, screenings and bookings may be maintained manually or across disconnected systems. This can result in duplicate data, difficulty in tracking bookings, inefficient searching and the possibility of double-booking seats.

The proposed system stores all cinema information in a structured MySQL database named **CinemaDB**. The database consists of six tables: **Movie, Screen, Customer, Seat, Screening and Booking**. Primary keys identify records uniquely, while foreign keys establish relationships between the tables. A unique constraint on `(screening\_id, seat\_id)` in the Booking table prevents the same seat from being booked more than once for a screening.

The project demonstrates database and table creation, insertion of sample data (five records in every table), and ten SQL queries that use joins, aggregate functions, grouping and filtering. The queries calculate the total confirmed revenue (₹1,300), the average ticket price (₹260), the highest ticket price (₹300), the number of bookings for each movie and the movies rated above 8.

A web-based user interface was designed to demonstrate a dashboard, booking management, seat selection, ticket confirmation and cancellation. The relational design is normalized up to the Third Normal Form (3NF) to reduce redundancy and avoid update anomalies. Overall, the project shows how a DBMS provides data consistency, integrity, organization and efficient access to cinema booking information.

Keywords: DBMS, MySQL, CinemaDB, ER diagram, normalization, SQL, JOIN, ticket booking.

TABLE OF CONTENTS

Certificate.....	
Declaration.....	
Acknowledgement	
Abstract	
Table of Contents	
1. Introduction and Problem Statement	
2. Objectives and Scope	
3. Existing System and Proposed System	
4. Software and Hardware Requirements	
5. System Architecture	
6. ER Diagram	
7. Relational Schema and Normalization.....	
8. Data Dictionary	
9. SQL Commands (DDL and DML) with Sample Outputs	
10. SQL Queries with Outputs.....	
11. UI Design and Screenshots	
12. Implementation Details	
13. Testing.....	
14. Conclusion and Future Enhancements.....	
15. References.....	
.....	

1. INTRODUCTION AND PROBLEM STATEMENT

1.1 Introduction

The entertainment industry handles a large amount of information related to movies, theatres, screenings, customers, seats and ticket bookings. Managing this information efficiently requires a reliable database system. A cinema ticket booking process includes several connected activities, and each activity depends on data produced by another.

First, information about movies must be maintained. Each movie has attributes such as title, genre, language, duration and rating. The cinema contains multiple screens, each having a particular capacity. Individual seats belong to screens and have seat numbers and seat types. A movie is scheduled for a particular date and time on a specific screen. Customers can then book available seats for a screening. Each booking records the customer, the screening, the seat, the booking date, the amount and the booking status.

The **Cinema Screening and Ticket Booking System** represents these operations in a relational database named **CinemaDB**. The project is developed for the Database Management Systems course and applies the concepts of database creation, keys, constraints, normalization, SQL queries, joins and aggregation to a realistic problem.

1.2 About CinemaDB

CinemaDB contains six relational tables. Relationships between the tables are maintained with foreign keys, and data integrity is protected by primary key, NOT NULL and UNIQUE constraints.

Table	Purpose	Records
Movie	Movie title, genre, language, duration and rating	5
Screen	Cinema screens and their capacity	5
Customer	Customer name, phone and email	5
Seat	Seats of each screen with seat type	5
Screening	Movie shows with date, time and screen	5
Booking	Tickets booked with amount and status	5

Table 1.1: Tables of CinemaDB

The project also includes SQL queries that retrieve useful information, for example screening details, complete booking information, the number of bookings for each movie, total revenue, average ticket price, premium seats and confirmed bookings.

1.3 Problem Statement

Managing cinema screenings and ticket bookings manually becomes difficult as the number of movies, customers, screens and bookings increases. A cinema must keep track of the movies currently available, their genres, languages, durations and ratings, the screens and their capacities, the seats of each screen, the screening dates and times, customer information, booked seats, ticket prices, booking status and the revenue generated by confirmed bookings.

A poorly designed system creates several problems:

- **Duplicate information:** customer, movie and screen details may be stored repeatedly for every booking.

- **Double booking:** without database constraints, two customers may book the same seat for the same screening.
- **Difficult data retrieval:** finding all customers who booked tickets or calculating total revenue is slow when data is maintained manually.
- **Data inconsistency:** if the same information is stored in several places, updating one copy and not the other leaves the data inconsistent.
- **Poor organization:** movie, customer, seat, screening and booking information should be stored separately but connected through meaningful relationships.

1.4 Proposed Solution

A centralized relational database solves these problems. Primary keys identify each record uniquely and foreign keys connect related tables, so that complete booking information can be retrieved with SQL joins. The `UNIQUE (screening\_id, seat\_id)` constraint in the Booking table prevents duplicate seat bookings for a screening. The system therefore provides a structured way to store and retrieve cinema information efficiently.

2. OBJECTIVES AND SCOPE

2.1 Objectives

The main objectives of the project are listed below.

Objective 1: Movie Management

To store and manage movie information including title, genre, language, duration and rating.

Objective 2: Screen Management

To maintain information about cinema screens and their capacities.

Objective 3: Seat Management

To associate seats with screens and identify seat types such as Regular and Premium.

Objective 4: Screening Management

To store movie screening schedules including date, time, movie and screen.

Objective 5: Customer Management

To maintain customer information including name, phone number and email.

Objective 6: Booking Management

To store customer bookings, selected seats, booking dates, ticket amounts and booking status, and to prevent duplicate seat booking for the same screening.

Objective 7: Data Integrity

To maintain consistency using primary keys, foreign keys, NOT NULL constraints, UNIQUE constraints and relationships.

Objective 8: Query Processing

To provide useful SQL queries for retrieving information and generating basic analytical results.

Objective 9: User Interface

To provide a user interface for database operations such as viewing, inserting and deleting records.

2.2 Scope of the Project

The scope of the project includes:

- Movie, screen, seat and customer management.
- Screening management and ticket booking management.
- Revenue calculation and booking analysis.
- Movie rating analysis and premium seat identification.
- A basic booking user interface for viewing, inserting and deleting records.

The project is primarily an academic DBMS implementation. It does not include a production payment gateway or live cinema infrastructure.

2.3 Intended Users

User	Role in the system
Cinema administrators	Maintain movies, screens, seats and screenings; review bookings and revenue
Booking staff	Create bookings for customers and cancel bookings when required
Customers	Select a movie, screening and seat and receive a booking confirmation

Table 2.1: Intended users

2.4 Limitations and Future Scope

The current version does not process online payments, does not include customer login and does not update seat availability in real time. These are listed as future enhancements in Chapter 14.

3. EXISTING AND PROPOSED SYSTEM

3.1 Existing System

In a basic manual cinema booking environment, information may be maintained in paper records, spreadsheets or separate applications. Such systems have several limitations:

1. Manual entry can cause errors.
2. Duplicate customer information may occur.
3. Seat availability may not be updated correctly.
4. Searching for booking information takes time.
5. Revenue calculation requires manual processing.
6. Maintaining relationships between customers, screenings and seats is difficult.
7. Double booking can occur if records are not synchronized.

3.2 Proposed System

The proposed CinemaDB system organizes all important cinema data into relational tables. The main chains of relationships are:

Chain	Meaning
Movie → Screening → Booking	A movie has many screenings and a screening has many bookings

Chain	Meaning
Screen → Seat → Booking	A screen has many seats and a seat can appear in many bookings (for different screenings)
Screen → Screening	A screen hosts many screenings
Customer → Booking	A customer can make many bookings

Table 3.1: Relationship chains in the proposed system

These relationships make it possible to retrieve complete booking information with SQL joins.

3.3 Comparison

Aspect	Existing system	Proposed system (CinemaDB)
Data storage	Paper or scattered files	Centralized relational database
Duplication	High	Reduced by separating data into tables
Consistency	Copies may disagree	Maintained by keys and constraints
Double booking	Possible	Prevented by UNIQUE (screening_id, seat_id)
Retrieval	Manual searching	SQL queries and joins
Reports	Manual calculation	SUM, AVG, MAX and COUNT queries

Table 3.2: Existing system versus proposed system

3.4 Advantages of the Proposed System

- Centralized data storage and reduced data duplication.
- Improved data consistency and integrity through constraints.
- Easy data retrieval and report generation.
- Efficient booking management and prevention of duplicate seat bookings.
- An extensible design that can support future features.

4. SOFTWARE AND HARDWARE REQUIREMENTS

4.1 Software Requirements

The project requires a relational database management system capable of executing SQL commands, and a web browser for the user interface.

Component	Technology	Use in the project
Database	MySQL 8.0	Stores CinemaDB and executes SQL
Front end	HTML, CSS, JavaScript	User interface for dashboard, bookings and seat selection
Database tool	MySQL Workbench / MySQL command line	Running the SQL script and queries
Editor	Visual Studio Code	Writing SQL and UI code
Browser	Any modern web browser	Running the user interface
Version control	Git / GitHub	Source code repository

Table 4.1: Software requirements

The SQL script uses the following commands and clauses:

Category	Commands
DDL	CREATE DATABASE, USE, DROP TABLE, CREATE TABLE, DESCRIBE
DML	INSERT, DELETE, SELECT
Clauses	WHERE, JOIN, LEFT JOIN, GROUP BY
Functions and operators	COUNT, SUM, AVG, MAX, UNION ALL

Table 4.2: SQL features used

4.2 Hardware Requirements

Item	Minimum	Recommended
Processor	Dual core or better	Modern Intel / AMD processor
RAM	4 GB	8 GB or higher
Storage	1 GB free	SSD storage
Display	1366 × 768	Full HD or better
Input	Keyboard and mouse	Keyboard and mouse
Network	Needed only for repository management	Broadband connection

Table 4.3: Hardware requirements

5. SYSTEM ARCHITECTURE

The project can be represented using a three-layer architecture consisting of a presentation layer, an application layer and a database layer.

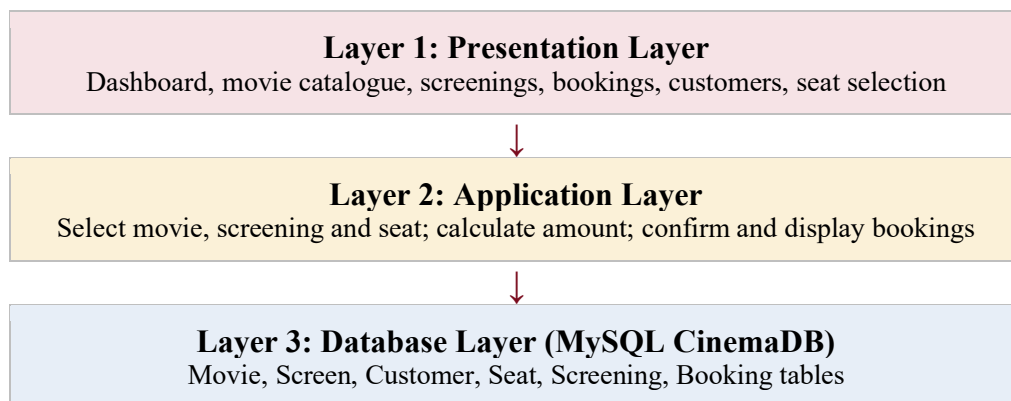


Figure 5.1: Three-layer architecture of the system

5.1 Presentation Layer

This is the user interface through which the administrator or booking staff interacts with the system. It includes the dashboard, the movie catalogue, screening information, booking records, customer records and the seat selection interface described in Chapter 11.

5.2 Application Layer

This layer contains the logic of the booking process: selecting movies, selecting screenings, selecting seats, calculating the booking amount, confirming bookings and displaying booking information.

5.3 Database Layer

The database layer stores persistent information in the six relational tables of CinemaDB. The tables Movie, Screening and Booking form the main chain, supported by the tables Customer, Screen and Seat.

5.4 Booking Data Flow

Step	Action	Database interaction
1	Customer selects a movie	SELECT from Movie
2	Customer selects a screening	SELECT from Screening joined with Movie and Screen
3	Available seats are displayed	SELECT from Seat; seats already in Booking for the screening are unavailable
4	Customer selects a seat	Seat identified by seat_id
5	Booking amount is calculated	Application logic
6	Booking is confirmed	INSERT into Booking with status Confirmed
7	Booking information is stored	Row stored; UNIQUE (screening_id, seat_id) enforced

Table 5.1: Booking data flow

6. ER DIAGRAM

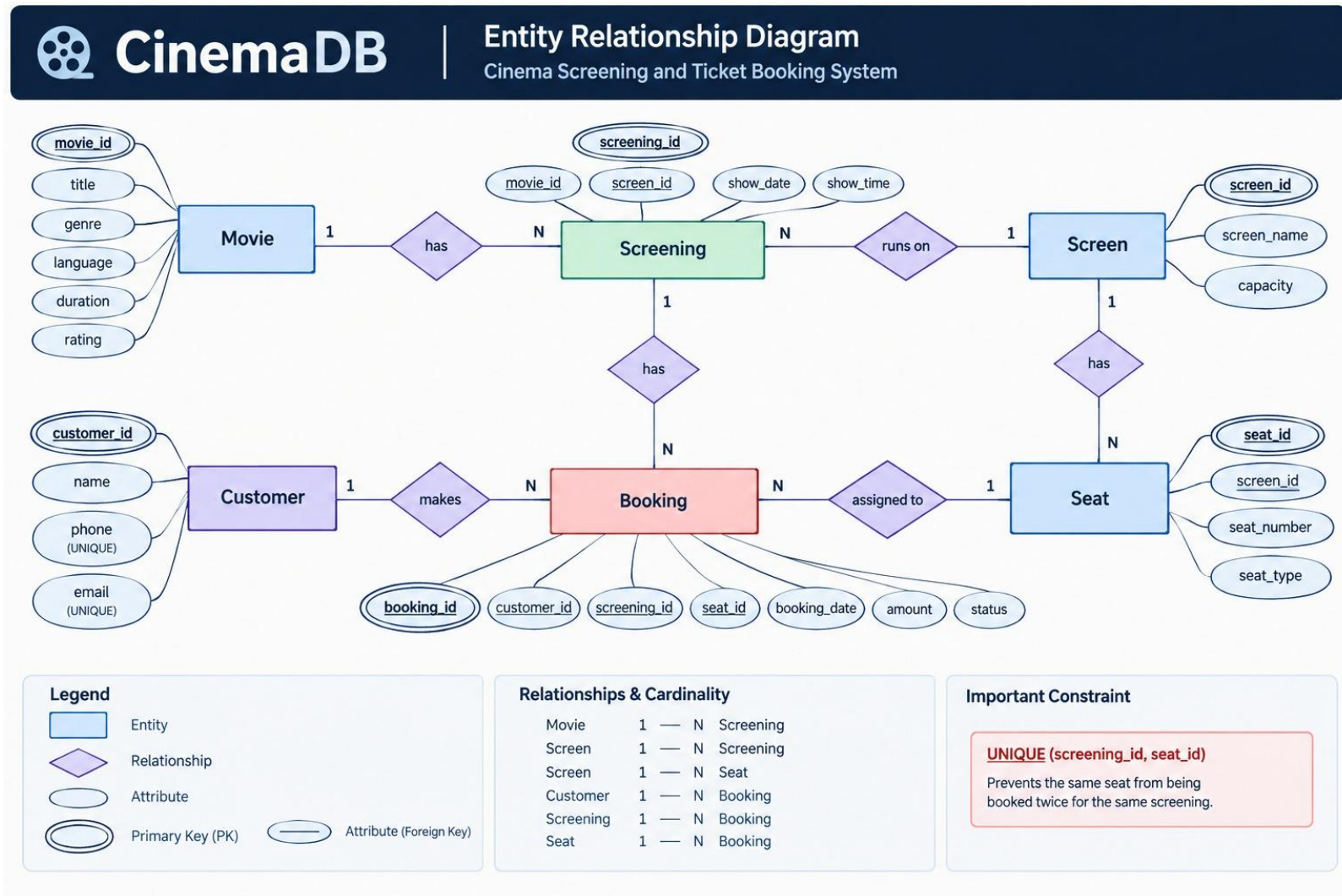


Figure 6.1: Entity Relationship Diagram of CinemaDB

6.1 Entities

The ER diagram contains exactly six entities. Primary keys are shown with double ellipses and foreign keys are underlined single ellipses in the diagram.

Entity	Primary key	Foreign keys	Other attributes
Movie	movie_id	—	title, genre, language, duration, rating
Screen	screen_id	—	screen_name, capacity
Customer	customer_id	—	name, phone, email
Seat	seat_id	screen_id	seat_number, seat_type
Screening	screening_id	movie_id, screen_id	show_date, show_time
Booking	booking_id	customer_id, screening_id, seat_id	booking_date, amount, status

Table 6.1: Entities and attributes

6.2 Relationships and Cardinality

Relationship	Entities	Cardinality	Participation	Implemented by
has	Movie – Screening	1 : M	Screening is total	Screening.movie_id
runs on	Screen – Screening	1 : M	Screening is total	Screening.screen_id
has	Screen – Seat	1 : M	Seat is total	Seat.screen_id
makes	Customer – Booking	1 : M	Booking is total	Booking.customer_id
has	Screening – Booking	1 : M	Booking is total	Booking.screening_id
assigned to	Seat – Booking	1 : M	Booking is total	Booking.seat_id

Table 6.2: Relationships of CinemaDB

Participation of the many-side entity is total because each foreign key is declared NOT NULL: every Screening must have a Movie and a Screen, every Seat must belong to a Screen, and every Booking must have a Customer, a Screening and a Seat. Participation of the one-side entity is partial, since a movie, customer or seat may exist without being referenced yet.

6.3 Important Constraints

The constraint `UNIQUE (screening\_id, seat\_id)` on the Booking table ensures that the same seat cannot be booked twice for the same screening. The constraint `UNIQUE (screen\_id, seat\_number)` on the Seat table ensures that a seat label is not repeated within the same screen. The columns screen_name, phone and email are also unique.

7. RELATIONAL SCHEMA AND NORMALIZATION

7.1 Relational Schema

The ER diagram is converted into the following relational schema. Primary keys are marked PK and foreign keys are marked FK.

Relation	Schema
Movie	Movie(movie_id PK, title, genre, language, duration, rating)

Relation	Schema
Screen	Screen(screen_id PK, screen_name, capacity)
Customer	Customer(customer_id PK, name, phone, email)
Seat	Seat(seat_id PK, screen_id FK, seat_number, seat_type)
Screening	Screening(screening_id PK, movie_id FK, screen_id FK, show_date, show_time)
Booking	Booking(booking_id PK, customer_id FK, screening_id FK, seat_id FK, booking_date, amount, status)

Table 7.1: Relational schema

7.2 Foreign Key References

Foreign key	References	Meaning
Seat.screen_id	Screen(screen_id)	Seat belongs to a screen
Screening.movie_id	Movie(movie_id)	Screening shows a movie
Screening.screen_id	Screen(screen_id)	Screening is held on a screen
Booking.customer_id	Customer(customer_id)	Booking made by a customer
Booking.screening_id	Screening(screening_id)	Booking is for a screening
Booking.seat_id	Seat(seat_id)	Booking reserves a seat

Table 7.2: Foreign key references

7.3 Candidate Keys and Unique Constraints

Table	Primary key	Additional unique constraints
Movie	movie_id	—
Screen	screen_id	screen_name
Customer	customer_id	phone; email
Seat	seat_id	(screen_id, seat_number)
Screening	screening_id	—
Booking	booking_id	(screening_id, seat_id)

Table 7.3: Keys and unique constraints

7.4 Normalization

Normalization is the process of organizing data to reduce redundancy and avoid insertion, update and deletion anomalies. CinemaDB is designed up to the **Third Normal Form (3NF)**.

Why normalization is needed

Suppose all information were stored in a single table with one row per booking, containing customer name, phone, movie title, genre, screen name, seat number and amount. The customer details would be repeated for every booking by that customer, the movie details would be repeated for every ticket sold for that movie, and changing a movie title would require updating many rows. Deleting the last booking of a movie would also remove all information about that movie. Splitting the data into the six tables removes these anomalies.

First Normal Form (1NF)

- All attributes contain atomic (indivisible) values.
- There are no repeating groups or multi-valued attributes.

- Every table has a primary key.

Every column in CinemaDB stores a single value. For example, a booking stores one seat_id; when a customer books several seats, one row is stored for each seat. Hence all six tables satisfy 1NF.

Second Normal Form (2NF)

- The table satisfies 1NF.
- Every non-key attribute depends on the complete primary key (no partial dependency).

All six tables use a single-column primary key (movie_id, screen_id, customer_id, seat_id, screening_id, booking_id). A partial dependency can occur only with a composite key, so no partial dependency exists and all tables satisfy 2NF.

Third Normal Form (3NF)

- The table satisfies 2NF.
- There is no transitive dependency.
- Non-key attributes depend only on the primary key.

In every table, each non-key attribute is a fact about the key and nothing else. For example, in Booking the customer's name is not stored; only customer_id is stored, and the name is obtained from Customer. Similarly, the movie title is not stored in Screening; movie_id refers to Movie. Therefore no transitive dependency exists.

Functional dependencies

Table	Functional dependency
Movie	movie_id → title, genre, language, duration, rating
Screen	screen_id → screen_name, capacity
Customer	customer_id → name, phone, email
Seat	seat_id → screen_id, seat_number, seat_type
Screening	screening_id → movie_id, screen_id, show_date, show_time
Booking	booking_id → customer_id, screening_id, seat_id, booking_date, amount, status

Table 7.4: Functional dependencies

Normalization summary

Normal form	Condition	Result
1NF	Atomic values, no repeating groups	Satisfied by all six tables
2NF	No partial dependency	Satisfied (single-column primary keys)
3NF	No transitive dependency	Satisfied by all six tables

Table 7.5: Normalization summary

8. DATA DICTIONARY

The data dictionary describes the structure, data types and constraints of every table in CinemaDB.

8.1 Table: Movie

Stores details of every movie shown in the cinema.

Column	Data type	Null	Key	Constraint	Description
movie_id	INT	NO	PRI	PRIMARY KEY	Unique identifier of the movie
title	VARCHAR(100)	NO		NOT NULL	Title of the movie
genre	VARCHAR(50)	NO		NOT NULL	Genre of the movie
language	VARCHAR(30)	NO		NOT NULL	Language of the movie
duration	INT	NO		NOT NULL	Running time in minutes
rating	DECIMAL(2,1)	YES		None (nullable)	Rating of the movie, for example 8.7

Table 8.1: Structure of the Movie table

8.2 Table: Screen

Stores the screens (auditoriums) of the cinema.

Column	Data type	Null	Key	Constraint	Description
screen_id	INT	NO	PRI	PRIMARY KEY	Unique identifier of the screen
screen_name	VARCHAR(50)	NO	UNI	NOT NULL, UNIQUE	Name of the screen, for example Screen 1
capacity	INT	NO		NOT NULL	Total seating capacity of the screen

Table 8.2: Structure of the Screen table

8.3 Table: Customer

Stores the customers who book tickets.

Column	Data type	Null	Key	Constraint	Description
customer_id	INT	NO	PRI	PRIMARY KEY	Unique identifier of the customer
name	VARCHAR(100)	NO		NOT NULL	Full name of the customer
phone	VARCHAR(15)	NO	UNI	NOT NULL, UNIQUE	Phone number of the customer
email	VARCHAR(100)	NO	UNI	NOT NULL, UNIQUE	Email address of the customer

Table 8.3: Structure of the Customer table

8.4 Table: Seat

Stores the individual seats belonging to each screen.

Column	Data type	Null	Key	Constraint	Description
seat_id	INT	NO	PRI	PRIMARY KEY	Unique identifier of the seat
screen_id	INT	NO	MUL	NOT NULL, FOREIGN KEY	Screen to which the seat belongs (Screen)
seat_number	VARCHAR(10)	NO		NOT NULL	Seat label, for example A1
seat_type	VARCHAR(20)	NO		NOT NULL	Seat category: Regular or Premium

Table 8.4: Structure of the Seat table

8.5 Table: Screening

Stores the scheduled shows of movies on screens.

Column	Data type	Null	Key	Constraint	Description
screening_id	INT	NO	PRI	PRIMARY KEY	Unique identifier of the screening
movie_id	INT	NO	MUL	NOT NULL, FOREIGN KEY	Movie being shown (Movie)
screen_id	INT	NO	MUL	NOT NULL, FOREIGN KEY	Screen on which the movie is shown (Screen)
show_date	DATE	NO		NOT NULL	Date of the show
show_time	TIME	NO		NOT NULL	Start time of the show

Table 8.5: Structure of the Screening table

8.6 Table: Booking

Stores the tickets booked by customers for screenings.

Column	Data type	Null	Key	Constraint	Description
booking_id	INT	NO	PRI	PRIMARY KEY	Unique identifier of the booking
customer_id	INT	NO	MUL	NOT NULL, FOREIGN KEY	Customer who made the booking (Customer)
screening_id	INT	NO	MUL	NOT NULL, FOREIGN KEY	Screening booked (Screening)
seat_id	INT	NO	MUL	NOT NULL, FOREIGN KEY	Seat booked (Seat)
booking_date	DATE	NO		NOT NULL	Date on which the booking was made
amount	DECIMAL(10,2)	NO		NOT NULL	Ticket amount in rupees
status	VARCHAR(20)	NO		NOT NULL	Booking status, for example Confirmed

Table 8.6: Structure of the Booking table

9. SQL COMMANDS (DDL AND DML)

9.1 Data Definition Language (DDL)

The database is created first and selected with USE. Old tables are dropped in reverse dependency order so that the script can be run repeatedly. Tables are then created in the order Movie, Screen, Customer, Seat, Screening and Booking, so that every referenced table already exists when a foreign key is declared.

Creating the database and removing old tables

```
CREATE DATABASE IF NOT EXISTS CinemaDB;
USE CinemaDB;

DROP TABLE IF EXISTS Booking;
DROP TABLE IF EXISTS Screening;
DROP TABLE IF EXISTS Seat;
DROP TABLE IF EXISTS Customer;
DROP TABLE IF EXISTS Screen;
DROP TABLE IF EXISTS Movie;
```

Listing 9.1: Database creation

Creating the tables

```
CREATE TABLE Movie (
    movie_id INT PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
```

```

    genre VARCHAR(50) NOT NULL,
    language VARCHAR(30) NOT NULL,
    duration INT NOT NULL,
    rating DECIMAL(2,1)
);

CREATE TABLE Screen (
    screen_id INT PRIMARY KEY,
    screen_name VARCHAR(50) NOT NULL UNIQUE,
    capacity INT NOT NULL
);

CREATE TABLE Customer (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(15) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE Seat (
    seat_id INT PRIMARY KEY,
    screen_id INT NOT NULL,
    seat_number VARCHAR(10) NOT NULL,
    seat_type VARCHAR(20) NOT NULL,
    FOREIGN KEY (screen_id) REFERENCES Screen(screen_id),
    UNIQUE (screen_id, seat_number)
);

CREATE TABLE Screening (
    screening_id INT PRIMARY KEY,
    movie_id INT NOT NULL,
    screen_id INT NOT NULL,
    show_date DATE NOT NULL,
    show_time TIME NOT NULL,
    FOREIGN KEY (movie_id) REFERENCES Movie(movie_id),
    FOREIGN KEY (screen_id) REFERENCES Screen(screen_id)
);

CREATE TABLE Booking (
    booking_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    screening_id INT NOT NULL,
    seat_id INT NOT NULL,
    booking_date DATE NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    status VARCHAR(20) NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (screening_id) REFERENCES Screening(screening_id),
    FOREIGN KEY (seat_id) REFERENCES Seat(seat_id),
    -- same seat cannot be booked twice for the same screening
    UNIQUE (screening_id, seat_id)
);

```

Listing 9.2: CREATE TABLE statements for the six tables

Table structure (DESCRIBE output)

The DESCRIBE command was run for each table to confirm the structure. The output for the Booking table is shown below.

```
mysql> DESCRIBE Booking;
```

Field	Type	Null	Key	Default	Extra
booking_id	int	NO	PRI	NULL	
customer_id	int	NO	MUL	NULL	
screening_id	int	NO	MUL	NULL	
seat_id	int	NO	MUL	NULL	
booking_date	date	NO		NULL	
amount	decimal(10,2)	NO		NULL	

Field	Type	Null	Key	Default	Extra
status	varchar(20)	NO		NULL	

Table 9.1: DESCRIBE Booking

9.2 Data Manipulation Language (DML)

Five records were inserted into each of the six tables. Parent tables are populated before child tables so that foreign key references are valid.

Inserting records

```

INSERT INTO Movie VALUES
(1, 'Interstellar', 'Science Fiction', 'English', 169, 8.7),
(2, 'Inception', 'Science Fiction', 'English', 148, 8.8),
(3, 'RRR', 'Action', 'Telugu', 187, 8.0),
(4, 'Kalki 2898 AD', 'Science Fiction', 'Telugu', 181, 7.0),
(5, '3 Idiots', 'Comedy Drama', 'Hindi', 170, 8.4);

INSERT INTO Screen VALUES
(1, 'Screen 1', 150), (2, 'Screen 2', 120), (3, 'Screen 3', 100),
(4, 'Screen 4', 180), (5, 'Screen 5', 200);

INSERT INTO Customer VALUES
(1, 'Aarav Sharma', '9876500001', 'aarav@gmail.com'),
(2, 'Ananya Reddy', '9876500002', 'ananya@gmail.com'),
(3, 'Rahul Kumar', '9876500003', 'rahul@gmail.com'),
(4, 'Sneha Patel', '9876500004', 'sneha@gmail.com'),
(5, 'Vikram Singh', '9876500005', 'vikram@gmail.com');

INSERT INTO Seat VALUES
(1, 1, 'A1', 'Regular'), (2, 1, 'A2', 'Regular'), (3, 2, 'B1', 'Premium'),
(4, 2, 'B2', 'Premium'), (5, 3, 'C1', 'Regular');

INSERT INTO Screening VALUES
(1, 1, 1, '2026-10-08', '10:00:00'),
(2, 2, 2, '2026-10-08', '13:00:00'),
(3, 3, 3, '2026-10-08', '16:00:00'),
(4, 4, 1, '2026-10-09', '19:00:00'),
(5, 5, 2, '2026-10-09', '21:00:00');

INSERT INTO Booking VALUES
(1, 1, 1, 1, '2026-10-06', 250.00, 'Confirmed'),
(2, 2, 2, 3, '2026-10-06', 300.00, 'Confirmed'),
(3, 3, 3, 5, '2026-10-06', 200.00, 'Confirmed'),
(4, 4, 4, 2, '2026-10-06', 250.00, 'Confirmed'),
(5, 5, 5, 4, '2026-10-06', 300.00, 'Confirmed');

```

Listing 9.3: INSERT statements

9.3 Sample Outputs of SELECT * Statements

After insertion, each table was displayed with a SELECT statement. Each table contains five records.

```
mysql> SELECT * FROM Movie;
```

movie_id	title	genre	language	duration	rating
1	Interstellar	Science Fiction	English	169	8.7
2	Inception	Science Fiction	English	148	8.8
3	RRR	Action	Telugu	187	8.0
4	Kalki 2898 AD	Science Fiction	Telugu	181	7.0
5	3 Idiots	Comedy Drama	Hindi	170	8.4

Table 9.2: Records of the Movie table

```
mysql> SELECT * FROM Screen;
```

screen_id	screen_name	capacity
1	Screen 1	150
2	Screen 2	120
3	Screen 3	100
4	Screen 4	180
5	Screen 5	200

Table 9.3: Records of the Screen table

```
mysql> SELECT * FROM Customer;
```

customer_id	name	phone	email
1	Aarav Sharma	9876500001	aarav@gmail.com
2	Ananya Reddy	9876500002	ananya@gmail.com
3	Rahul Kumar	9876500003	rahul@gmail.com
4	Sneha Patel	9876500004	sneha@gmail.com
5	Vikram Singh	9876500005	vikram@gmail.com

Table 9.4: Records of the Customer table

```
mysql> SELECT * FROM Seat;
```

seat_id	screen_id	seat_number	seat_type
1	1	A1	Regular
2	1	A2	Regular
3	2	B1	Premium
4	2	B2	Premium
5	3	C1	Regular

Table 9.5: Records of the Seat table

```
mysql> SELECT * FROM Screening;
```

screening_id	movie_id	screen_id	show_date	show_time
1	1	1	2026-10-08	10:00:00
2	2	2	2026-10-08	13:00:00
3	3	3	2026-10-08	16:00:00
4	4	1	2026-10-09	19:00:00
5	5	2	2026-10-09	21:00:00

Table 9.6: Records of the Screening table

```
mysql> SELECT * FROM Booking;
```

booking_id	customer_id	screening_id	seat_id	booking_date	amount	status
1	1	1	1	2026-10-06	250.00	Confirmed
2	2	2	3	2026-10-06	300.00	Confirmed
3	3	3	5	2026-10-06	200.00	Confirmed
4	4	4	2	2026-10-06	250.00	Confirmed
5	5	5	4	2026-10-06	300.00	Confirmed

Table 9.7: Records of the Booking table

9.4 INSERT and DELETE Operations for the UI Demonstration

An insert and a delete operation were also executed to demonstrate that data can be added and removed. A new customer was inserted, verified, deleted and verified again.

```
INSERT INTO Customer (customer_id, name, phone, email)
VALUES (6, 'Harshitha Rani', '9876500006', 'harshitha@gmail.com');
```

```
SELECT * FROM Customer WHERE customer_id = 6;

DELETE FROM Customer WHERE customer_id = 6;

SELECT * FROM Customer WHERE customer_id = 6;
```

Listing 9.4: INSERT and DELETE demonstration

Step	Result
After INSERT	customer_id 6 Harshitha Rani 9876500006 harshitha@gmail.com (1 row)
After DELETE	Empty set (the record is removed; Customer again has 5 records)

Table 9.8: Result of the INSERT and DELETE demonstration

9.5 Final Record Count

```
SELECT 'Movie' AS Table_Name, COUNT(*) AS Total_Records FROM Movie
UNION ALL SELECT 'Screen', COUNT(*) FROM Screen
UNION ALL SELECT 'Customer', COUNT(*) FROM Customer
UNION ALL SELECT 'Seat', COUNT(*) FROM Seat
UNION ALL SELECT 'Screening', COUNT(*) FROM Screening
UNION ALL SELECT 'Booking', COUNT(*) FROM Booking;
```

Table_Name	Total_Records
Movie	5
Screen	5
Customer	5
Seat	5
Screening	5
Booking	5

Table 9.9: Record count of every table

10. SQL QUERIES WITH OUTPUTS

Ten queries were executed on CinemaDB to retrieve useful information. The queries use JOIN, LEFT JOIN, GROUP BY, WHERE and the aggregate functions COUNT, SUM, AVG and MAX. The output tables show the results for the sample data of Chapter 9.

10.1 Query 1: Movies and screening details

A JOIN between Movie and Screening displays each movie with its show date and time.

```
SELECT m.title, m.genre, s.show_date, s.show_time
FROM Movie m
JOIN Screening s ON m.movie_id = s.movie_id;
```

title	genre	show_date	show_time
Interstellar	Science Fiction	2026-10-08	10:00:00
Inception	Science Fiction	2026-10-08	13:00:00
RRR	Action	2026-10-08	16:00:00
Kalki 2898 AD	Science Fiction	2026-10-09	19:00:00
3 Idiots	Comedy Drama	2026-10-09	21:00:00

Table 10.1: Output of Query 1

10.2 Query 2: Complete booking details

Multiple JOINS combine Booking, Customer, Screening, Movie, Screen and Seat to display complete booking information.

```
SELECT c.name AS customer_name, m.title AS movie, sc.screen_name,
       se.seat number, b.booking date, b.amount, b.status
FROM Booking b
JOIN Customer c ON b.customer_id = c.customer_id
JOIN Screening s ON b.screening_id = s.screening_id
JOIN Movie m ON s.movie id = m.movie id
JOIN Screen sc ON s.screen id = sc.screen id
JOIN Seat se ON b.seat_id = se.seat_id;
```

customer_name	movie	screen_name	seat_number	booking_date	amount	status
Aarav Sharma	Interstellar	Screen 1	A1	2026-10-06	250.00	Confirmed
Ananya Reddy	Inception	Screen 2	B1	2026-10-06	300.00	Confirmed
Rahul Kumar	RRR	Screen 3	C1	2026-10-06	200.00	Confirmed
Sneha Patel	Kalki 2898 AD	Screen 1	A2	2026-10-06	250.00	Confirmed
Vikram Singh	3 Idiots	Screen 2	B2	2026-10-06	300.00	Confirmed

Table 10.2: Output of Query 2

10.3 Query 3: Count of bookings for each movie

GROUP BY with COUNT gives the number of bookings for each movie. A LEFT JOIN keeps movies that have no booking.

```
SELECT m.title, COUNT(b.booking_id) AS total_bookings
FROM Movie m
JOIN Screening s ON m.movie id = s.movie id
LEFT JOIN Booking b ON s.screening_id = b.screening_id
GROUP BY m.movie_id, m.title;
```

title	total_bookings
Interstellar	1
Inception	1
RRR	1
Kalki 2898 AD	1
3 Idiots	1

Table 10.3: Output of Query 3

10.4 Query 4: Total confirmed revenue

SUM adds the amounts of all confirmed bookings: 250 + 300 + 200 + 250 + 300.

```
SELECT SUM(amount) AS total_revenue
FROM Booking
WHERE status = 'Confirmed';
```

total_revenue
1300.00

Table 10.4: Output of Query 4

Total confirmed revenue = ₹1,300.

10.5 Query 5: Average ticket price

AVG calculates the average amount of the bookings.

```
SELECT AVG(amount) AS average_ticket_price
FROM Booking;
```

average_ticket_price
260.000000

Table 10.5: Output of Query 5

10.6 Query 6: Highest ticket price

MAX returns the highest ticket amount.

```
SELECT MAX(amount) AS highest_ticket_price
FROM Booking;
```

highest_ticket_price
300.00

Table 10.6: Output of Query 6

10.7 Query 7: Customers who booked tickets

A JOIN between Customer and Booking lists the customers who have booked tickets.

```
SELECT c.name, c.email, b.amount, b.status
FROM Customer c
JOIN Booking b ON c.customer_id = b.customer_id;
```

name	email	amount	status
Aarav Sharma	aarav@gmail.com	250.00	Confirmed
Ananya Reddy	ananya@gmail.com	300.00	Confirmed
Rahul Kumar	rahul@gmail.com	200.00	Confirmed
Sneha Patel	sneha@gmail.com	250.00	Confirmed
Vikram Singh	vikram@gmail.com	300.00	Confirmed

Table 10.7: Output of Query 7

10.8 Query 8: Movies with rating above 8

The WHERE clause filters movies whose rating is greater than 8.0. RRR (8.0) is not included because 8.0 is not greater than 8.0.

```
SELECT title, genre, rating
FROM Movie
WHERE rating > 8.0;
```

title	genre	rating
Interstellar	Science Fiction	8.7
Inception	Science Fiction	8.8
3 Idiots	Comedy Drama	8.4

Table 10.8: Output of Query 8

10.9 Query 9: Premium seats

A JOIN between Seat and Screen with a WHERE condition lists the premium seats and their screens.

```
SELECT se.seat number, se.seat type, sc.screen name
FROM Seat se
JOIN Screen sc ON se.screen_id = sc.screen_id
WHERE se.seat_type = 'Premium';
```

seat_number	seat_type	screen_name
B1	Premium	Screen 2
B2	Premium	Screen 2

Table 10.9: Output of Query 9

10.10 Query 10: Confirmed bookings

The WHERE clause displays only the bookings whose status is Confirmed.

```
SELECT *
FROM Booking
WHERE status = 'Confirmed';
```

booking_id	customer_id	screening_id	seat_id	booking_date	amount	status
1	1	1	1	2026-10-06	250.00	Confirmed
2	2	2	3	2026-10-06	300.00	Confirmed
3	3	3	5	2026-10-06	200.00	Confirmed
4	4	4	2	2026-10-06	250.00	Confirmed
5	5	5	4	2026-10-06	300.00	Confirmed

Table 10.10: Output of Query 10

10.11 Summary of Results

Query	Technique	Result
1	JOIN	5 movie screenings listed
2	Multiple JOINS	5 complete booking records
3	COUNT, GROUP BY	1 booking for each movie
4	SUM	₹1,300 total confirmed revenue
5	AVG	₹260 average ticket price
6	MAX	₹300 highest ticket price
7	JOIN	5 customers who booked tickets
8	WHERE	3 movies rated above 8
9	JOIN, WHERE	2 premium seats (B1, B2)
10	WHERE	5 confirmed bookings

Table 10.11: Summary of query results

11. UI DESIGN AND SCREENSHOTS

11.1 Design Overview

A web-based user interface named **Cinema DB** was designed using HTML, CSS and JavaScript. The interface uses a dark theme with red accents. The left navigation menu provides the sections Dashboard, Movies, Screenings, Bookings, Customers and Screens, together with Reports and DBMS Schema under the heading Analytics and DBMS. The interface supports viewing records, inserting a new booking and deleting (cancelling) a booking.

UI screen	Purpose	Database operation
Dashboard	Summary cards, now-showing movies, upcoming screenings and recent bookings	VIEW
Bookings	Searchable list of all bookings with status	VIEW
New Booking	Three steps: Screening, Seat Selection, Confirmation	INSERT
Ticket confirmation	Shows the confirmed booking and ticket	INSERT result
Cancel booking	Cancels a booking from the Bookings list	DELETE

Table 11.1: User interface screens

11.2 Dashboard

The dashboard opens after login as administrator. The top row shows summary cards for total revenue, total bookings, active movies, active screens and occupancy rate. Below them, the Now Showing Movies panel lists each movie with its genre, language, duration and rating, and a Book Tickets button. The Upcoming Screenings panel shows the show time, screen and the seats left.

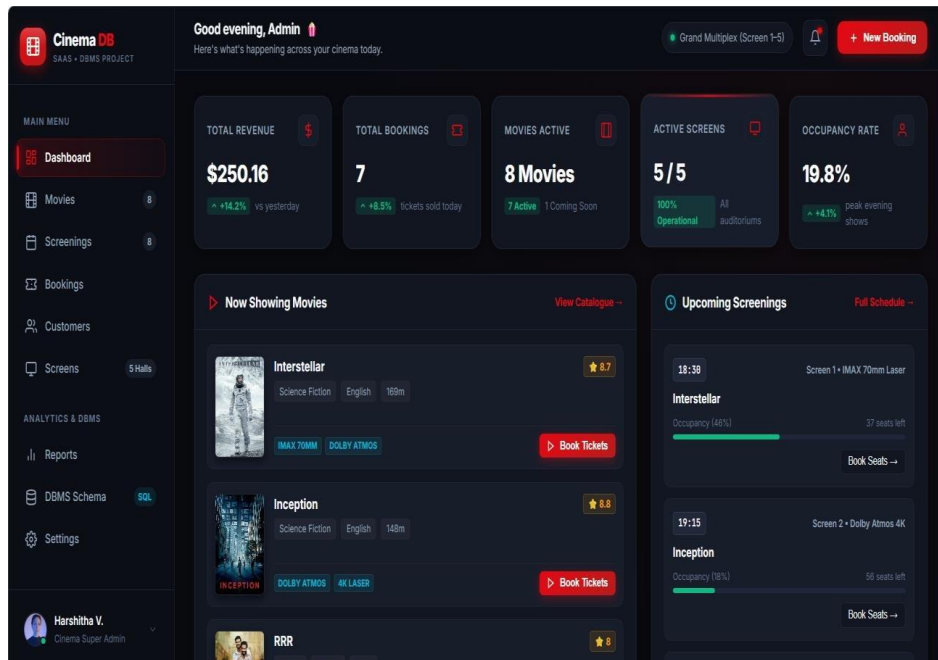


Figure 11.1: Dashboard with summary cards, now showing movies and upcoming screenings

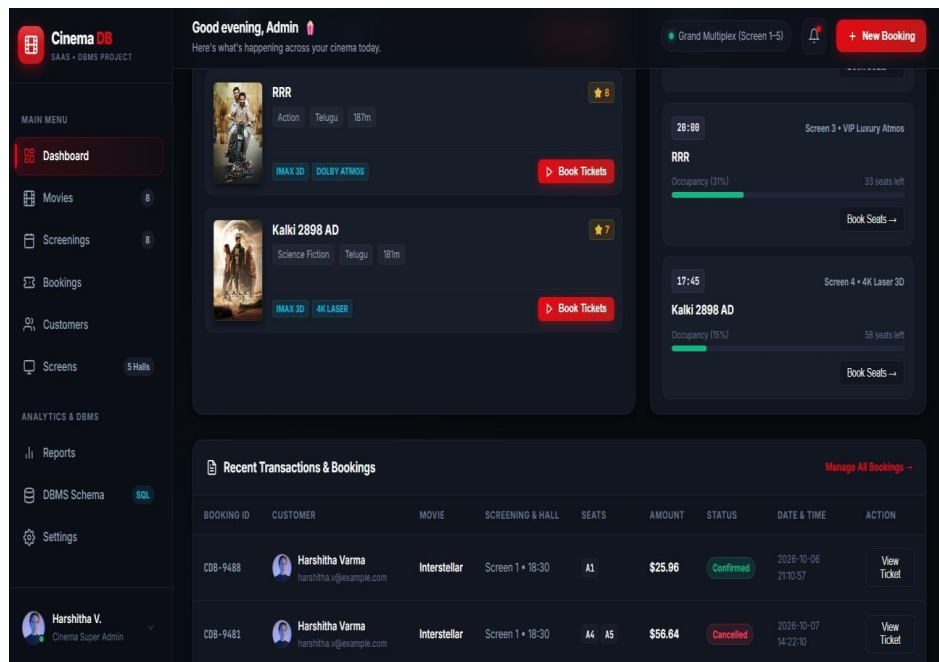


Figure 11.2: Dashboard (scrolled) showing more movies and screenings

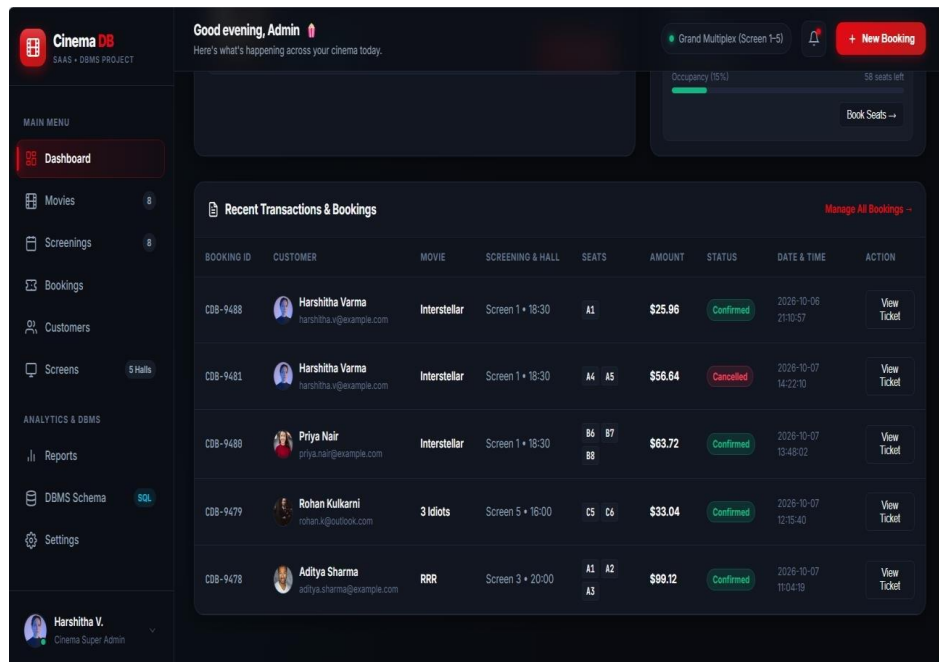
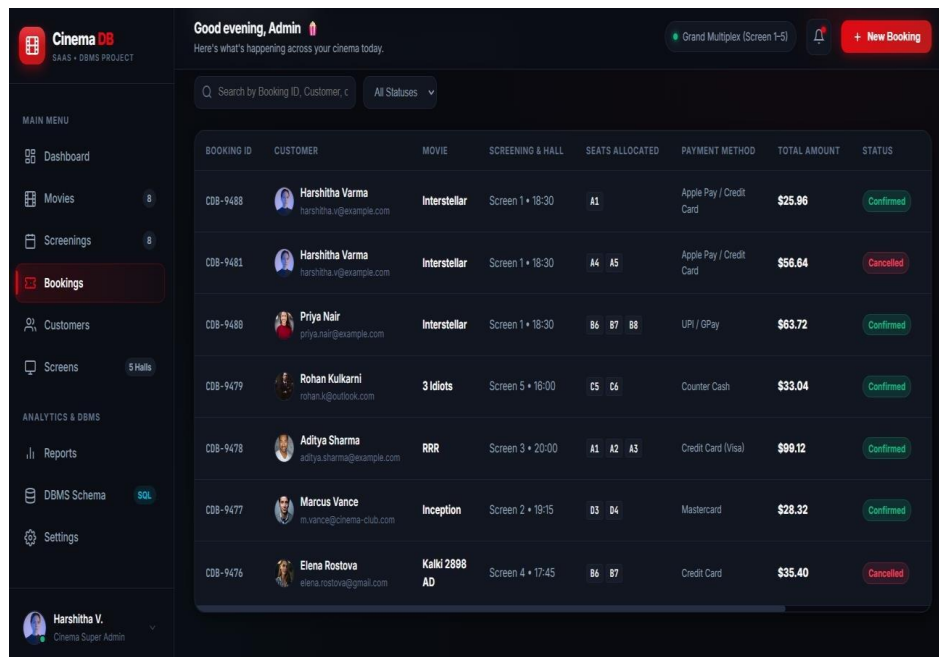


Figure 11.3: Recent Transactions and Bookings table on the dashboard

The Recent Transactions and Bookings table lists the booking ID, customer, movie, screening and hall, seats, amount, status and date and time of each booking, with an action button to view the ticket.

11.3 Bookings Screen (Before Insertion)

The Bookings screen displays all bookings with a search box for booking ID or customer and a filter for the booking status. The screenshot below shows the list before a new booking is inserted; the latest booking at this point is CDB-9488.



BOOKING ID	CUSTOMER	MOVIE	SCREENING & HALL	SEATS ALLOCATED	PAYMENT METHOD	TOTAL AMOUNT	STATUS
CDB-9488	Harshitha Varma harshitha.v@example.com	Interstellar	Screen 1 • 18:30	A1	Apple Pay / Credit Card	\$25.96	Confirmed
CDB-9481	Harshitha Varma harshitha.v@example.com	Interstellar	Screen 1 • 18:30	A4 A5	Apple Pay / Credit Card	\$56.64	Cancelled
CDB-9489	Priya Nair priya.n@example.com	Interstellar	Screen 1 • 18:30	B6 B7 B8	UPI / GPay	\$63.72	Confirmed
CDB-9479	Rohan Kulkarni rohan.k@outlook.com	3 Idiots	Screen 5 • 16:00	C5 C6	Counter Cash	\$33.04	Confirmed
CDB-9478	Aditya Sharma aditya.sharma@example.com	RRR	Screen 3 • 20:00	A1 A2 A3	Credit Card (Visa)	\$99.12	Confirmed
CDB-9477	Marcus Vance m.vance@cinema-club.com	Inception	Screen 2 • 19:15	B3 B4	Mastercard	\$28.32	Confirmed
CDB-9476	Elena Rostova elena.rostova@gmail.com	Kalki 2898 AD	Screen 4 • 17:45	B6 B7	Credit Card	\$35.40	Cancelled

Figure 11.4: Bookings list before insertion

11.4 Insertion: New Booking and Seat Selection

A new booking is created with the New Booking button. The booking wizard has three steps: Screening, Seat Selection and Confirmation. In the seat selection step, a hall map shows available, selected, booked and VIP recliner seats, grouped as VIP Recliner, Club Premium and Executive Standard. A booking summary on the right shows the movie, the selected seats and the customer details.

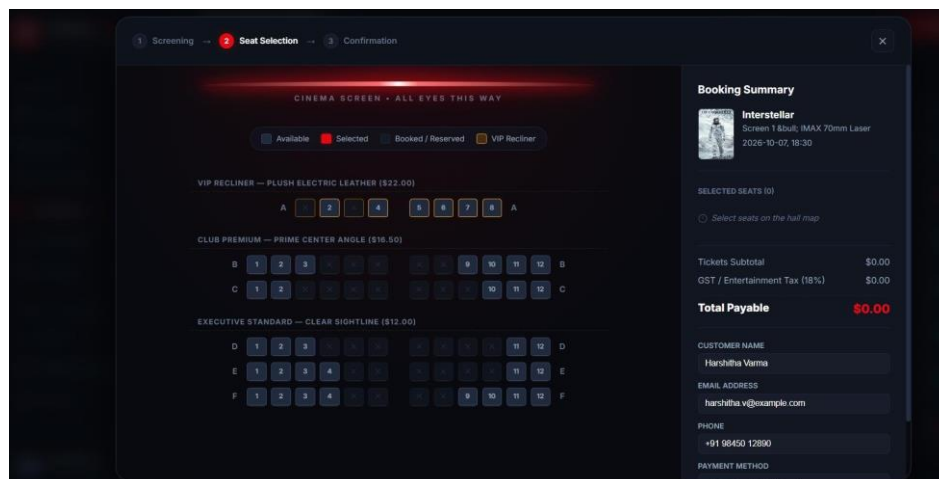


Figure 11.5: Seat selection step of the booking wizard

11.5 Booking Confirmation (After Insertion)

After the booking is confirmed, an e-ticket is displayed with the booking number CDB-9489, the movie Interstellar, the auditorium Screen 1, date 2026-10-07, show time 18:30 and the allocated seats, together with a confirmation message that the seats are reserved.

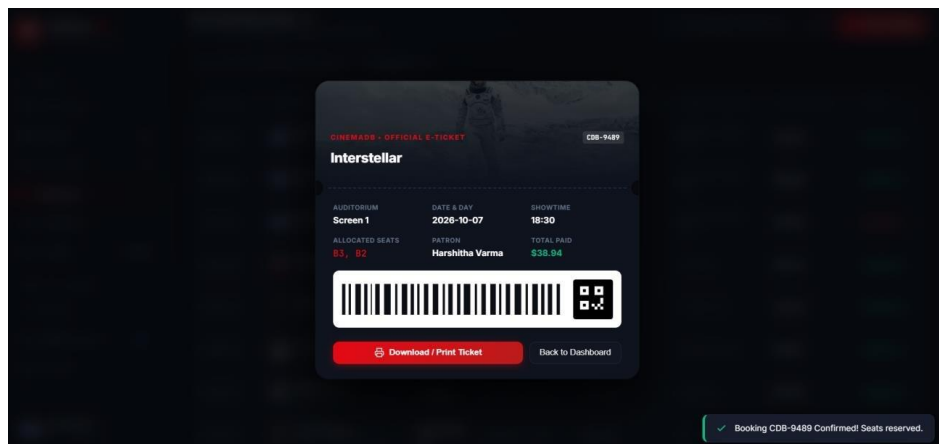


Figure 11.6: Ticket confirmation after a new booking is inserted

The screenshot shows the 'Bookings' list in the Cinema DB application. The table lists bookings with columns: Booking ID, Customer, Movie, Screening & Hall, Seats Allocated, Payment Method, Total Amount, and Status. The new booking CDB-9489 is highlighted in red.

BOOKING ID	CUSTOMER	MOVIE	SCREENING & HALL	SEATS ALLOCATED	PAYMENT METHOD	TOTAL AMOUNT	STATUS
CDB-9489	Harshitha Varma	Interstellar	Screen 1 • 18:30	B3, B2	Apple Pay / Credit Card	\$38.94	Confirmed
CDB-9488	Harshitha Varma	Interstellar	Screen 1 • 18:30	A1	Apple Pay / Credit Card	\$25.96	Confirmed
CDB-9481	Harshitha Varma	Interstellar	Screen 1 • 18:30	A4, A5	Apple Pay / Credit Card	\$56.64	Cancelled
CDB-9480	Priya Nair	Interstellar	Screen 1 • 18:30	B6, B7, B8	UPI / GPay	\$63.72	Confirmed
CDB-9479	Rohan Kulkarni	3 Idiots	Screen 5 • 16:00	C5, C6	Counter Cash	\$33.04	Confirmed
CDB-9478	Aditya Sharma	RRR	Screen 3 • 20:00	A1, A2, A3	Credit Card (Visa)	\$99.12	Confirmed
CDB-9477	Marcus Vance	Inception	Screen 2 • 19:15	D3, D4	Mastercard	\$28.32	Confirmed
CDB-9476	Elena Rostova	Kalki 2898 AD	Screen 4 • 17:45	B6, B7	Credit Card	\$35.40	Cancelled

Figure 11.7: Bookings list after insertion, showing the new booking CDB-9489

11.6 Deletion: Cancelling a Booking

A booking can be cancelled from the Actions column of the Bookings list. The first screenshot shows the list before cancellation, with a Cancel button for each active booking. The second shows the result after booking CDB-9488 is cancelled; its status changes to Cancelled and the Cancel button is no longer shown.

The screenshot shows the 'Bookings' list in the Cinema DB application, including an 'ACTIONS' column with 'Ticket' and 'Cancel' buttons. The booking CDB-9488 is highlighted in red.

MOVIE	SCREENING & HALL	SEATS ALLOCATED	PAYMENT METHOD	TOTAL AMOUNT	STATUS	TRANSACTION AT	ACTIONS
Interstellar	Screen 1 • 18:30	B3, B2	Apple Pay / Credit Card	\$38.94	Confirmed	2026-10-07 05:48:18	Ticket Cancel
Interstellar	Screen 1 • 18:30	A1	Apple Pay / Credit Card	\$25.96	Confirmed	2026-10-06 21:50:57	Ticket Cancel
Interstellar	Screen 1 • 18:30	A4, A5	Apple Pay / Credit Card	\$56.64	Cancelled	2026-10-07 14:22:16	Ticket
Interstellar	Screen 1 • 18:30	B6, B7, B8	UPI / GPay	\$63.72	Confirmed	2026-10-07 13:48:50	Ticket Cancel
3 Idiots	Screen 5 • 16:00	C5, C6	Counter Cash	\$33.04	Confirmed	2026-10-07 12:15:40	Ticket Cancel
RRR	Screen 3 • 20:00	A1, A2, A3	Credit Card (Visa)	\$99.12	Confirmed	2026-10-07 10:04:19	Ticket Cancel
Inception	Screen 2 • 19:15	D3, D4	Mastercard	\$28.32	Confirmed	2026-10-07 09:30:19	Ticket Cancel
Kalki 2898 AD	Screen 4 • 17:45	B6, B7	Credit Card	\$35.40	Cancelled	2026-10-06 19:40:00	Ticket

Figure 11.8: Bookings list before deletion (cancellation)

MOVIE	SCREENING & HALL	SEATS ALLOCATED	PAYMENT METHOD	TOTAL AMOUNT	STATUS	TRANSACTED AT	ACTIONS
Interstellar	Screen 1 • 18:30	B3 B2	Apple Pay / Credit Card	\$38.94	Confirmed	2023-10-07 03:40:18	Ticket Cancel
Interstellar	Screen 1 • 18:30	A1	Apple Pay / Credit Card	\$25.96	Cancelled	2023-10-08 21:09:57	Ticket
Interstellar	Screen 1 • 18:30	A4 A5	Apple Pay / Credit Card	\$56.64	Cancelled	2023-10-07 14:22:10	Ticket
Interstellar	Screen 1 • 18:30	B6 B7 B8	UPI / GPay	\$63.72	Confirmed	2023-10-07 13:43:02	Ticket Cancel
3 Idiots	Screen 5 • 16:00	C5 C6	Counter Cash	\$33.04	Confirmed	2023-10-07 12:19:45	Ticket Cancel
RRR	Screen 3 • 20:00	A1 A2 A3	Credit Card (Visa)	\$90.12	Confirmed	2023-10-07 11:04:19	Ticket Cancel
Inception	Screen 2 • 19:15	D3 D4	Mastercard	\$28.32	Confirmed	2023-10-07 09:30:16	Ticket Cancel
Kalki 2898 AD	Screen 4 • 17:45	B6 B7	Credit Card	\$35.40	Cancelled	2023-10-08 19:40:00	Ticket

Figure 11.9: Bookings list after deletion (cancellation) with confirmation message

11.7 Note on the UI Prototype

The user interface is a design prototype. It displays presentation elements such as payment method, tax, ticket prices and poster images that are not stored in the six tables of CinemaDB. The data stored permanently in the database is limited to the attributes of the six tables described in Chapter 8.

12. IMPLEMENTATION DETAILS

12.1 Technology Stack

Layer	Technology
Database	MySQL 8.0 (database CinemaDB)
Front end	HTML, CSS and JavaScript
Tools	MySQL Workbench, Visual Studio Code, web browser, Git and GitHub

Table 12.1: Technology stack

12.2 Database Implementation

The database was implemented with a single SQL script that creates the database, drops old tables, creates the six tables with their constraints, inserts the sample data, displays the tables and runs the queries. The script is submitted as a separate .sql file. Referential integrity is maintained by the FOREIGN KEY constraints, which prevent a booking from referencing a customer, screening or seat that does not exist.

12.3 Database Connectivity and UI Operations

The user interface performs three kinds of operations on the database: viewing, inserting and deleting records. Each operation corresponds to an SQL statement executed on CinemaDB, and a change made through the interface is reflected in the database tables.

UI action	SQL statement executed
View bookings	SELECT with JOINS on Booking, Customer, Screening, Movie, Screen and Seat
View movies and screenings	SELECT with JOIN on Movie and Screening
View customers	SELECT * FROM Customer

UI action	SQL statement executed
Confirm a new booking (insert)	INSERT INTO Booking (booking_id, customer_id, screening_id, seat_id, booking_date, amount, status) VALUES (...)
Cancel or delete a record	DELETE FROM <table> WHERE <primary key> = <value>

Table 12.2: UI operations and the corresponding SQL

12.4 Key SQL Snippets

The two most important implementation features are the foreign keys that connect the tables and the unique constraint that prevents double booking:

```
FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
FOREIGN KEY (screening_id) REFERENCES Screening(screening_id),
FOREIGN KEY (seat_id) REFERENCES Seat(seat_id),

-- Prevent the same seat from being booked twice for the same screening
UNIQUE (screening_id, seat_id)
```

Listing 12.1: Constraints of the Booking table

The query that retrieves complete booking details joins six tables and is used by the Bookings view:

```
FROM Booking b
JOIN Customer c ON b.customer_id = c.customer_id
JOIN Screening s ON b.screening_id = s.screening_id
JOIN Movie m ON s.movie_id = m.movie_id
JOIN Screen sc ON s.screen_id = sc.screen_id
JOIN Seat se ON b.seat_id = se.seat_id;
```

Listing 12.2: JOINS used to retrieve complete booking information

12.5 Integrity Rules Enforced by the Database

- **Entity integrity:** every table has a primary key that cannot be NULL or duplicated.
- **Referential integrity:** every foreign key value must exist in the parent table; a parent row that is still referenced cannot be deleted.
- **Domain integrity:** data types and NOT NULL constraints restrict the values of each column.
- **Uniqueness:** screen_name, phone, email, (screen_id, seat_number) and (screening_id, seat_id) cannot be repeated.

13. TESTING

The system was tested at the database level and at the user interface level. Each test case lists the action performed, the expected result and the observed result.

13.1 Test Cases

ID	Test case	Action	Expected result	Result
T1	Database connection	Connect to MySQL and run USE CinemaDB	Database is selected	Pass
T2	Table creation	Run the CREATE TABLE statements	Six tables are created without error	Pass
T3	Record insertion	Run the INSERT statements	Five records in each table	Pass
T4	SELECT queries	SELECT * on every table	All five rows are displayed	Pass
T5	JOIN queries	Run Queries 1, 2, 7 and 9	Correct joined rows are displayed	Pass
T6	Aggregate queries	Run Queries 3 to 6	COUNT = 1 per movie, SUM = 1300, AVG = 260, MAX = 300	Pass

ID	Test case	Action	Expected result	Result
T7	INSERT operation	Insert customer 6	Row appears in Customer	Pass
T8	DELETE operation	Delete customer 6	Row is removed from Customer	Pass
T9	Foreign key constraint	Insert a booking with a non-existing customer_id	Insert rejected with a foreign key error	Pass
T10	Duplicate seat booking	Insert a second booking for the same screening_id and seat_id	Insert rejected with a duplicate entry error	Pass

Table 13.1: Database test cases

13.2 Constraint Tests

The two most important constraint tests are shown below. The first attempts to book seat 1 again for screening 1, which is already booked by booking 1. The second attempts to create a booking for a customer that does not exist.

```
-- T10: duplicate seat for the same screening
INSERT INTO Booking VALUES (6, 2, 1, 1, '2026-10-06', 250.00, 'Confirmed');
-- Rejected: duplicate entry for UNIQUE (screening_id, seat_id)

-- T9: non-existing customer
INSERT INTO Booking VALUES (7, 99, 2, 4, '2026-10-06', 300.00, 'Confirmed');
-- Rejected: foreign key constraint fails
```

Listing 13.1: Constraint test statements

13.3 User Interface Test Cases

ID	Test case	Expected result	Result
U1	View dashboard and bookings	Records are displayed correctly	Pass
U2	Create a new booking (insert)	Booking CDB-9489 is confirmed and appears in the Bookings list	Pass
U3	Cancel booking CDB-9488 (delete)	Status changes to Cancelled and a confirmation message is shown	Pass
U4	Seat selection	Selected seats appear in the booking summary	Pass

Table 13.2: User interface test cases

13.4 Testing Summary

All test cases passed. The constraints of the database correctly prevented invalid data: a seat cannot be booked twice for the same screening, and a booking cannot refer to a customer, screening or seat that does not exist.

14. CONCLUSION AND FUTURE ENHANCEMENTS

14.1 Conclusion

The **Cinema Screening and Ticket Booking System** provides a structured relational database for managing movies, screens, seats, customers, screenings and bookings. The database CinemaDB was designed with six tables, converted from an ER diagram with six one-to-many relationships, and normalized up to the Third Normal Form.

Primary keys, foreign keys, NOT NULL and UNIQUE constraints maintain the integrity of the data. In particular, the constraint `UNIQUE (screening\_id, seat\_id)` prevents duplicate seat bookings. Ten SQL queries demonstrated joins, aggregate functions, grouping and filtering and produced useful information

such as the total confirmed revenue of ₹1,300, the average ticket price of ₹260 and the highest ticket price of ₹300. A user interface was designed to demonstrate viewing, inserting and deleting records.

The project helped the team understand how the theoretical concepts of DBMS – ER modeling, relational design, normalization, SQL and constraints – are applied to a practical problem.

14.2 Future Enhancements

Enhancement	Description
Online payment integration	Accept payments online and store the payment status of each booking
Customer login	Authentication so that customers can view their own bookings
Real-time seat selection	Update seat availability instantly as seats are selected
Email / SMS confirmation	Send booking confirmations to customers
Advanced analytics	Reports and dashboards such as revenue by movie and screen occupancy
Role-based access control	Different permissions for administrators, staff and customers

Table 14.1: Future enhancements

15. REFERENCES

1. A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, McGraw-Hill Education.
2. R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, Pearson.
3. R. Ramakrishnan and J. Gehrke, Database Management Systems, McGraw-Hill Education.
4. MySQL 8.0 Reference Manual, Oracle Corporation.
<https://dev.mysql.com/doc/refman/8.0/en/>
5. DBMS course lecture notes and project guidelines, School of Technology, Woxsen University.